



CAREERSENSE ACADEMY

PROFESSIONAL LEARNING HANDBOOK

Git & GitHub for Professionals

Version control, collaboration, pull requests, recovery, release workflows,
and team-scale engineering habits

For software engineers • data professionals • AI builders • DevOps engineers • technical product teams

CareerSense Professional Series • careersenseai.com/resources

How to Use This Handbook

Git is not just a set of commands, and GitHub is not just a place to store code. Together they form a professional collaboration system: a way to make changes safely, explain intent, review work, recover from mistakes, and move software from an idea to a trusted release.

This handbook focuses on the mental models and workflows professionals use every day. You will still type commands, but every command is tied to a reason: what state is changing, what can be recovered, what teammates will see, and how the history will help someone later.

What each chapter contains

Each chapter follows a practical rhythm: context, learning objectives, core concepts, command-line examples, GitHub workflow examples, common mistakes, professional habits, revision notes, practice questions, and hands-on exercises.

PROFESSIONAL FOCUS

The goal is not to memorize every Git command. The goal is to become the teammate who can make changes confidently, keep history understandable, collaborate through pull requests, and recover when something goes wrong.

Recommended practice environment

- Create one disposable repository specifically for this book.
- Run `git status` after almost every command until state changes feel natural.
- Use `git log --oneline --graph --decorate --all` to visualize history.
- Push to a private or test GitHub repository for pull-request exercises.
- Intentionally create a conflict and resolve it; recovery is a skill, not an accident.

CONTENTS

What You Will Learn

#	Chapter
1	Git Mental Model: Working Tree, Staging Area, Repository
2	Commits, History, and Professional Change Sets
3	Branches, Merges, Rebases, and Conflict Resolution
4	GitHub Collaboration: Remotes, Pull Requests, and Reviews
5	Undo, Recovery, Reflog, and Safe Troubleshooting
6	Team Workflows: Trunk-Based, Feature Branches, and Release Flow
7	Tags, Releases, GitHub Actions, and Repository Hygiene
8	CareerSense Mini Project: Ship a Collaborative Feature End to End

Closing the book: whole-book revision • professional Git glossary • 20-question quiz • answer key

01

Git Mental Model: Working Tree, Staging Area, Repository

Git becomes much easier when you stop seeing commands as magic and start seeing three states: the files you are editing, the snapshot you are preparing, and the committed history you have saved.

LEARNING OBJECTIVES

- Explain the working tree, staging area, and repository
- Initialize and inspect a repository
- Stage changes intentionally
- Understand tracked and untracked files
- Read git status as a state report
- Use .gitignore without hiding important files

Main Concepts

A local Git repository has three practical layers. The working tree is what is on disk right now. The staging area, also called the index, is the snapshot you are preparing for the next commit. The repository is the history of committed snapshots stored inside the hidden .git directory. Most everyday Git commands move information between these layers.

State	What it represents	Common action
Working tree	Current files on disk	Edit, create, delete
Staging area	Changes selected for the next commit	git add
Repository	Saved commit history	git commit

TERMINAL

```
mkdir careersense-demo
```

```
cd careersense-demo
git init
printf "# CareerSense Demo\n" > README.md
git status
```

PROFESSIONAL HABIT

Treat git status as your dashboard. Before committing, switching branches, rebasing, or resolving a conflict, read it first. It tells you what Git believes is happening.

Staging is a design tool

Staging is not an annoying extra step. It lets you separate unrelated edits into clean commits. If you fixed a typo while building a feature, you can stage only the feature changes first and keep the typo for a second commit.

TERMINAL

```
git add README.md
git diff --staged
git commit -m "docs: add project overview"
```

.gitignore and repository boundaries

TERMINAL

```
printf ".env\n.venv\n__pycache__\n*.log\n" > .gitignore
git add .gitignore
git commit -m "chore: ignore local environment files"
```

COMMON MISTAKE

Adding secrets first and then placing them in .gitignore. .gitignore prevents future tracking; it does not erase data already committed. Rotate exposed secrets and remove them from history when necessary.

QUICK REVISION NOTES

- Working tree = current files.
- Staging area = next proposed snapshot.
- Repository = committed history.
- git status explains state; git diff explains content changes.
- Use staging to create focused commits.
- .gitignore is preventive, not a secret-removal tool.

Practice Questions

- Q1** What are the three main local Git states?
- Q2** Why is staging useful in professional work?
- Q3** What is the difference between `git diff` and `git diff --staged`?
- Q4** Why can `.gitignore` not protect a secret that was already committed?

Hands-on Exercises

- 1** Create a repository and make one untracked file, one staged file, and one committed file.
- 2** Use `git add -p` to stage only part of a modified file.
- 3** Create a `.gitignore` for a Python project and verify ignored files do not appear in `git status`.

02

Commits, History, and Professional Change Sets

A professional commit is not merely a checkpoint. It is a small explanation of intent that future reviewers, teammates, debuggers, and automation can understand.

LEARNING OBJECTIVES

- Create focused commits
- Write useful commit messages
- Inspect history with log and show
- Compare commits and branches
- Use amend safely
- Understand commit identity and hashes

A commit is a snapshot plus context

Each commit records a snapshot of tracked content, metadata such as author and time, a message, and a parent link to previous history. The commit hash identifies that exact object. Change the content or metadata and the hash changes too.

GOOD COMMIT TEST

A reviewer should be able to answer “what changed and why?” from the commit title and diff. If a commit mixes refactoring, feature work, formatting, and unrelated fixes, the answer becomes harder.

Message structure

TERMINAL

```
git commit -m "feat: add application status filter"  
git commit -m "fix: preserve referral source on edit"  
git commit -m "docs: explain local setup"
```

Teams may use Conventional Commits or another scheme, but the deeper rule is stable: prefer imperative, specific messages that describe the outcome, not the typing you performed.

Reading history

TERMINAL

```
git log --oneline --graph --decorate --all
git show HEAD
git diff HEAD~1..HEAD
```

COMMON MISTAKE

Using `git commit --amend` after you have already pushed a commit that teammates may depend on. Amend rewrites the commit identity. It is usually safe before sharing, but coordinate before rewriting shared history.

When to amend

Amend is excellent for fixing the most recent local commit: adding a forgotten file, correcting a message, or cleaning a change before push. After shared history exists, prefer a new commit unless your team explicitly allows rewriting.

QUICK REVISION NOTES

- Commits are immutable objects identified by hashes.
- Focused commits improve review and recovery.
- Commit messages should describe outcome and intent.
- `git log` shows history; `git show` inspects one commit.
- `git commit --amend` rewrites the latest commit.
- Avoid casually rewriting shared history.

Practice Questions

- Q1** What information does a commit contain?
- Q2** What makes a commit “focused”?
- Q3** When is amend generally safe?
- Q4** Why does changing an amended commit create a new hash?

Hands-on Exercises

- 1 Create three focused commits from one larger set of edits.
- 2 Use `git show` to inspect the exact patch of the second commit.
- 3 Amend a local commit message and compare the old and new hashes.

03

Branches, Merges, Rebases, and Conflict Resolution

Branches let teams explore change without freezing everyone else. Merges and rebases are two different ways to bring histories together, and conflicts are simply places where Git needs a human decision.

LEARNING OBJECTIVES

- Create and switch branches
- Merge branches
- Explain fast-forward and merge commits
- Rebase local work
- Resolve conflicts deliberately
- Choose merge vs rebase based on collaboration needs

Branches are movable names

A Git branch is a lightweight name pointing to a commit. Creating a branch does not duplicate the repository. As new commits are made on that branch, the pointer moves forward.

TERMINAL

```
git switch -c feature/interview-score
# edit files
git add .
git commit -m "feat: calculate interview readiness score"
```

Merge: preserve both lines of history

TERMINAL

```
git switch main
git merge feature/interview-score
```

If main has not moved, Git may fast-forward the branch pointer. If both branches have new commits, Git can create a merge commit that records both parents.

Rebase: replay commits on a new base

TERMINAL

```
git switch feature/interview-score
git fetch origin
git rebase origin/main
```

HISTORY RULE

Rebase is excellent for cleaning your own unshared branch. Avoid rebasing commits that other people have already based work on unless your team has a clear coordination process.

Conflict resolution

TERMINAL

```
git status
# open conflicted file and choose the correct final content
git add path/to/file
git rebase --continue
```

RESULT

The conflict is resolved only when the file contains the intended final result and the resolution is staged.

COMMON MISTAKE

Resolving a conflict by blindly choosing “ours” or “theirs.” A conflict is a product decision as much as a text decision. Understand what each side intended before choosing the final content.

QUICK REVISION NOTES

- Branches are pointers, not copies.
- Merge combines histories; rebase replays commits.
- Fast-forward is possible when histories have not diverged.
- Rebase rewrites commit identities.
- Conflicts require an intentional final file state.
- Use git status throughout conflict resolution.

Practice Questions

- Q1** What does a branch point to?
- Q2** How does merge differ from rebase?
- Q3** Why does rebase change commit hashes?
- Q4** What should you do before accepting one side of a conflict?

Hands-on Exercises

- 1 Create two branches that edit different files and merge them.
- 2 Create a deliberate same-line conflict and resolve it.
- 3 Rebase a local feature branch onto a newer main and inspect the graph before and after.

04

GitHub Collaboration: Remotes, Pull Requests, and Reviews

Git handles version history locally. GitHub adds shared hosting, pull requests, review conversations, permissions, automation, and a collaborative record of how a change became trusted.

LEARNING OBJECTIVES

- Connect local repositories to remotes
- Fetch, pull, and push intentionally
- Open effective pull requests
- Review code with useful comments
- Use protected branches and checks
- Understand forks vs branches

Remote mental model

A remote such as origin is simply a named location for another Git repository. Remote-tracking names like origin/main represent your local record of what that remote branch looked like the last time you fetched.

TERMINAL

```
git remote -v
git fetch origin
git log --oneline --graph --decorate --all
```

Command	Purpose	Professional note
git fetch	Download remote objects and refs	Safest way to inspect remote change without integrating it
git pull	Fetch then integrate	Know whether your team uses merge or rebase behavior
git push	Publish local commits	May trigger CI, reviews, or

		deployments
--	--	-------------

Pull request anatomy

- A concise title that states the outcome.
- Context: what problem or user need is being addressed.
- What changed: the important implementation decisions.
- How tested: commands, automated checks, or manual scenarios.
- Risks or rollout notes.
- Screenshots or evidence when behavior is visual.

REVIEW MINDSET

A pull request is not just permission to merge. It is a durable explanation of a change. Write it so a teammate six months later can understand why the decision was made.

Review comments that help

- Ask about correctness and edge cases before style preferences.
- Distinguish blocking issues from optional suggestions.
- Explain why a change matters instead of only giving an instruction.
- Acknowledge good design decisions as well as problems.
- Keep architectural debates visible when they affect future maintainers.

COMMON MISTAKE

Using git pull reflexively without knowing how it integrates changes. A pull can create a merge commit or invoke rebasing depending on configuration. Fetch first when you want to inspect before integrating.

QUICK REVISION NOTES

- A remote is another repository location.
- fetch downloads without integrating.
- pull fetches and integrates.
- push publishes commits.
- Pull requests combine code, discussion, checks, and approval.
- Good reviews focus on correctness, clarity, maintainability, and risk.

Practice Questions

- Q1** What is origin/main?
- Q2** How does fetch differ from pull?
- Q3** What information belongs in a strong pull request?
- Q4** What makes a review comment actionable?

Hands-on Exercises

- 1 Push a feature branch to GitHub and open a pull request.
- 2 Add a pull-request template with Context, Testing, and Risk sections.
- 3 Review a sample diff and write one blocking comment and one non-blocking suggestion.

05

Undo, Recovery, Reflog, and Safe Troubleshooting

Git is powerful partly because most local mistakes are recoverable. The professional skill is knowing whether you want to restore file content, unstage a change, move a branch, create a reversing commit, or recover a commit you thought was lost.

LEARNING OBJECTIVES

- Distinguish restore, reset, and revert
- Unstage without deleting work
- Undo committed changes safely
- Recover commits with reflog
- Use stash for temporary context switches
- Troubleshoot from state instead of panic

Different mistakes need different tools

Situation	Safer first tool	Why
Discard unstaged edits	<code>git restore <file></code>	Restores working-tree content
Unstage a file	<code>git restore --staged <file></code>	Keeps working-tree edits
Undo a shared commit	<code>git revert <commit></code>	Adds a new reversing commit
Move local branch backward	<code>git reset</code>	Rewrites local branch position
Find a “lost” commit	<code>git reflog</code>	Shows recent HEAD movements

TERMINAL

```
git restore --staged config.yaml
git status
```

Revert for shared history

TERMINAL

```
git revert a1b2c3d
```

Revert records a new commit whose patch reverses the selected commit. Because existing history remains intact, it is usually the preferred undo mechanism after changes have been shared.

Reflog: your local safety net

TERMINAL

```
git reflog
# find the previous HEAD, then inspect it
git show HEAD@{2}
# recover by creating a branch
git branch recovery/work HEAD@{2}
```

RECOVERY HABIT

When something looks lost, stop making destructive changes. Run `git status`, `git log --all --oneline --graph`, and `git reflog`. Recovery becomes harder only when more state is changed blindly.

Stash carefully

TERMINAL

```
git stash push -m "wip: partial dashboard filters"
git switch hotfix/production
# later
git stash list
git stash pop
```

COMMON MISTAKE

Using `git reset --hard` as a universal cleanup command. It can destroy uncommitted work. Read status and choose the narrowest undo command that matches the state you actually want to change.

QUICK REVISION NOTES

- restore changes file or staging state.
- revert safely undoes shared commits by adding history.
- reset moves branch pointers and can discard local state.
- reflog records recent local HEAD movements.
- stash is temporary storage, not a long-term branch strategy.
- Troubleshoot by inspecting state before changing it.

Practice Questions

- Q1** Why is revert safer than reset for shared history?
- Q2** What does reflog record?
- Q3** How do you unstage while keeping file edits?
- Q4** Why is reset --hard risky?

Hands-on Exercises

- 1** Create a commit, reset it locally, then recover it with reflog.
- 2** Revert a test commit and inspect both commits in the log.
- 3** Stash work, switch branches, then restore the stash.

06

Team Workflows: Trunk-Based, Feature Branches, and Release Flow

A Git workflow is a team agreement about how change moves from an idea to mainline code. The best workflow is not the one with the most branches; it is the one that keeps integration frequent, ownership clear, and recovery predictable.

LEARNING OBJECTIVES

- Compare common team workflows
- Use short-lived branches
- Keep main releasable
- Understand release and hotfix branches
- Reduce merge risk with frequent integration
- Define branch protection and ownership rules

Common workflow patterns

Workflow	Typical shape	Strength
Trunk-based	Very short branches or direct integration to main	Fast integration and small changes
Feature branch	One branch per change, merged via PR	Clear review boundary
Release branch	Stabilization branch near a release	Supports controlled release hardening
Fork workflow	Contributors push to personal forks	Useful for open source and external contributors

TEAM PRINCIPLE

Branch lifetime matters more than branch naming. A feature branch that lives for three weeks

accumulates integration risk even if the naming convention is perfect.

A practical feature-branch flow

TERMINAL

```
git switch main
git pull --ff-only
git switch -c feature/cover-letter-export
# small commits
git push -u origin feature/cover-letter-export
# open PR, review, checks, merge
```

Keeping main healthy

- Require automated checks before merge.
- Prefer small pull requests that are easy to review.
- Use feature flags for incomplete behavior when appropriate.
- Define who can bypass branch protection and why.
- Document emergency hotfix procedures before an emergency happens.

COMMON MISTAKE

Treating a long-lived development branch as a safe buffer while main falls behind. Large delayed integrations create difficult conflicts and hide integration bugs.

QUICK REVISION NOTES

- Workflows are social contracts backed by Git.
- Short-lived branches reduce integration risk.
- Main should remain releasable whenever practical.
- Feature branches provide review boundaries.
- Release branches can stabilize a specific release line.
- Branch protection makes team expectations enforceable.

Practice Questions

- Q1** What is the main advantage of trunk-based development?
- Q2** Why are long-lived branches risky?
- Q3** When might a release branch be useful?
- Q4** What does branch protection accomplish?

Hands-on Exercises

- 1 Write a one-page Git workflow for a five-person product team.
- 2 Design branch naming and PR rules for an AI application.

3 List the checks you would require before merging to main.

07

Tags, Releases, GitHub Actions, and Repository Hygiene

Professional repositories do more than hold source code. They communicate versions, automate checks, document ownership, and provide predictable paths from commit to release.

LEARNING OBJECTIVES

- Create and push tags
- Understand annotated vs lightweight tags
- Create GitHub releases
- Read a basic GitHub Actions workflow
- Use repository metadata effectively
- Maintain clean, secure repositories

Tags mark important commits

TERMINAL

```
git tag -a v1.4.0 -m "CareerSense interview simulator v1.4.0"
git push origin v1.4.0
```

Annotated tags store tag metadata and are commonly used for releases. A tag should point to the exact commit you intend to identify as a version.

A simple GitHub Actions check

YAML

```
name: tests

on:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
- uses: actions/setup-python@v5
  with:
    python-version: "3.12"
- run: pip install -r requirements.txt
- run: pytest
```

AUTOMATION PRINCIPLE

CI should provide fast evidence about a change: tests, linting, security checks, builds, or policy validation. A green check is useful only when the check represents something the team genuinely cares about.

Repository hygiene checklist

- README with purpose, setup, and contribution basics.
- LICENSE when distribution terms matter.
- CONTRIBUTING.md for contribution expectations.
- CODEOWNERS where review ownership should be explicit.
- Issue and pull-request templates for consistency.
- Dependabot or equivalent dependency update strategy.
- Secret scanning and protected branches.
- Release notes or changelog strategy.

COMMON MISTAKE

Committing generated files, local IDE state, credentials, or large binaries without a deliberate repository policy. Repository cleanliness affects clone speed, security, review quality, and long-term maintainability.

QUICK REVISION NOTES

- Tags identify specific commits.
- Annotated tags are common for releases.
- GitHub Releases add release notes and assets around tags.
- Actions automate checks and workflows.
- Repository metadata reduces onboarding friction.
- Security and hygiene are part of version-control professionalism.

Practice Questions

- Q1** What is the purpose of a tag?
- Q2** What does a GitHub Release add beyond a tag?
- Q3** What is the value of CI on pull requests?
- Q4** Name four repository-hygiene files or settings.

Hands-on Exercises

- 1 Tag a test repository with v0.1.0.
- 2 Create a GitHub Actions workflow that runs one simple check.
- 3 Draft a README outline and pull-request template for a professional repository.

08

CareerSense Mini Project: Ship a Collaborative Feature End to End

The final project combines local Git, GitHub collaboration, review, recovery, and release thinking into one realistic workflow: delivering a small CareerSense feature from issue to tagged release.

LEARNING OBJECTIVES

- Plan a change from an issue
- Create a branch and focused commits
- Open and update a pull request
- Handle review feedback
- Resolve a conflict safely
- Merge and create a release tag

Scenario

The CareerSense application needs a new “application source” field so users can record whether a job came from LinkedIn, a referral, a company website, a recruiter, or another source. Your task is to ship the change using a professional Git and GitHub workflow.

Step 1 - Create the issue and acceptance criteria

ISSUE BRIEF

Add application source to the job tracker. Acceptance criteria: user can select a source, value persists, analytics can group by source, existing records remain valid, tests cover the new behavior.

Step 2 - Branch from updated main

TERMINAL

```
git switch main
git fetch origin
git pull --ff-only
git switch -c feature/application-source
```

Step 3 - Build in focused commits

TERMINAL

```
git add database/migrations
git commit -m "feat: add application source field"

git add app tests
git commit -m "feat: capture source in job tracker"

git add docs
git commit -m "docs: explain application source analytics"
```

Step 4 - Push and open a pull request

TERMINAL

```
git push -u origin feature/application-source
```

- Title: “Add application source tracking”.
- Explain the user problem.
- List migration and backward-compatibility decisions.
- Describe automated and manual tests.
- Add screenshots if the UI changed.

Step 5 - Incorporate review

Suppose a reviewer asks for an “Unknown” fallback so historical records remain analyzable. Make the change in a focused commit and push again; the pull request updates automatically.

TERMINAL

```
git add app tests
git commit -m "fix: default missing application source to unknown"
git push
```

Step 6 - Resolve an integration conflict

TERMINAL

```
git fetch origin
git rebase origin/main
# resolve conflict
git add app/models.py
git rebase --continue
git push --force-with-lease
```

WHY FORCE-WITH-LEASE

After rebasing your own branch, the remote branch history must be updated. `--force-with-lease` refuses to overwrite remote work you have not seen, making it safer than a blind `--force`.

Step 7 - Merge and release

TERMINAL

```
git switch main
git pull --ff-only
git tag -a v1.5.0 -m "Application source analytics"
git push origin v1.5.0
```

Professional reflection

- Was each commit independently understandable?
- Did the pull request explain intent and testing?
- Were conflicts resolved based on desired behavior rather than text convenience?
- Could another engineer recover the decision history six months later?
- Did the release tag identify the exact production-ready commit?

QUICK REVISION NOTES

- Issue -> branch -> focused commits -> push -> PR -> review -> checks -> merge -> release.
- Use `--force-with-lease` after rebasing a private feature branch.
- Keep PR context and testing evidence visible.
- Treat conflicts as design decisions.
- Tags mark release commits.

Practice Questions

- Q1** Why is `--force-with-lease` safer than `--force`?
- Q2** What should an issue define before coding starts?
- Q3** How do focused commits improve review?
- Q4** What information should remain discoverable after the feature ships?

Hands-on Exercises

- 1 Repeat the project with a different small feature.
- 2 Create a release note for v1.5.0.
- 3 Write a post-merge checklist your team could reuse.

Course Summary

Professional Git usage is a combination of state awareness, history design, collaboration discipline, and recovery confidence. You began with the three-state mental model, then learned to build focused commits and read history. Branches introduced parallel work; merges, rebases, and conflict resolution showed how histories come back together.

GitHub added the shared layer: remotes, pull requests, reviews, branch protection, and automation. Recovery tools such as restore, revert, reset, stash, and reflog gave you ways to undo the right thing without damaging unrelated work. Team workflows connected these mechanics to delivery, while tags, releases, Actions, and repository hygiene turned a code repository into a professional engineering system.

The final project connected all of those habits into one path from issue to release. That is the skill to carry forward: not “knowing Git commands,” but being able to move a change through a team safely, transparently, and recoverably.

Quick Revision Notes - The Whole Book

QUICK REVISION NOTES

- Working tree, staging area, and repository are the core local states.
- Focused commits make history reviewable and reversible.
- Branches are pointers; merge combines histories; rebase replays commits.
- Conflicts require a human decision about the correct final content.
- fetch downloads; pull downloads and integrates; push publishes.
- Pull requests preserve context, review, checks, and decisions.
- restore, revert, reset, stash, and reflog solve different recovery problems.
- Short-lived branches reduce integration risk.
- Tags identify release commits; GitHub Releases add notes and assets.
- GitHub Actions automate evidence such as tests and builds.
- Repository hygiene includes documentation, ownership, protection, and secret safety.

REFERENCE

Professional Git & GitHub Glossary

Term	Meaning in plain English
Working tree	The files currently checked out on disk.
Staging area / index	The proposed snapshot for the next commit.
Repository	The .git data store containing objects, references, and history.
Commit	An immutable snapshot object with metadata and parent links.
HEAD	The reference describing the commit or branch currently checked out.
Branch	A movable name pointing to a commit.
Remote	A named location for another Git repository.
origin	The conventional default name for the first remote.
Remote-tracking branch	A local reference such as origin/main recording the last fetched remote state.
Merge	Combining histories, sometimes with a merge commit.
Rebase	Replaying commits on a different base, producing new commit identities.
Conflict	A case where Git cannot automatically determine the intended combined content.
Pull request	A GitHub collaboration object for reviewing and integrating a proposed branch change.
Review	Structured feedback and approval around a change.
Revert	A new commit that reverses the patch of an earlier

	commit.
Reset	Moving a branch reference and optionally changing staging/working-tree state.
Restore	Replacing working-tree or staged content from another Git state.
Reflog	Local log of recent reference and HEAD movements.
Stash	Temporary storage for uncommitted working changes.
Tag	A name, usually fixed, identifying a particular commit.
Release	A GitHub object built around a tag with notes and optional assets.
CI	Automated checks that run against code changes.
GitHub Actions	GitHub workflow automation defined in YAML.
Branch protection	Repository rules controlling how protected branches can be changed.
CODEOWNERS	A file that maps paths to responsible reviewers.
force-with-lease	A guarded force-push that checks the remote branch has not changed unexpectedly.

Professional Practice Quiz

1. Which area represents the exact snapshot prepared for the next commit?
2. Which command is the best first dashboard for repository state?
3. What does `git diff --staged` compare?
4. Why should commits be focused?
5. What does a branch point to?
6. What does rebase do conceptually?
7. Why can rebasing shared history be disruptive?
8. What does `git fetch` do?
9. What is a pull request primarily for?
10. Why is filter placement important with LEFT JOIN-like repository thinking?
11. Which Git command is usually preferred to undo a shared commit?
12. Where can you often find a commit after an accidental local reset?
13. What is the danger of `git reset --hard`?
14. What is the main risk of long-lived feature branches?
15. What does branch protection enforce?
16. What does an annotated tag commonly represent?
17. What is GitHub Actions used for?
18. Why is `--force-with-lease` safer than `--force`?
19. What should a good pull request explain besides the code diff?
20. What is the professional goal of Git mastery?

ANSWER KEY

Quiz Answers

1. Staging area / index
2. git status
3. The staged snapshot against the current commit
4. To improve review, understanding, and safe recovery
5. A commit
6. Replays commits on a new base
7. It rewrites commit identities that others may reference
8. Downloads remote objects and refs without integrating them
9. Collaborative review and integration of a proposed change
10. N/A - trick question; joins are SQL, not Git. The lesson is to recognize domain boundaries.
11. git revert
12. git reflog
13. It can discard uncommitted working-tree changes
14. Delayed integration and larger conflicts
15. Repository rules such as required reviews and checks
16. A named release or important version commit
17. Automating workflows such as tests, builds, and checks
18. It refuses to overwrite unexpected remote changes
19. Context, intent, testing, risk, and important decisions
20. Move changes through a team safely, transparently, and recoverably



CareerSense Academy

Build skills. Prove them. Move your career forward.

More learning resources: careersenseai.com/resources

NEXT PRACTICE

Use Git on a real project this week. Make one small branch, create focused commits, open a pull request, ask for review, and inspect the history after merge. Professional fluency comes from repeated, deliberate use.